

Tuning3

April 11, 2025

```
[ ]: !pip install keras-tuner --upgrade

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf
import keras_tuner as kt

from tensorflow.keras.layers import Bidirectional, LSTM, Dropout, Dense
from sklearn.preprocessing import MinMaxScaler, StandardScaler

#####
# 1) Utility Functions for Data Preparation
#####
def compute_rsi(series, period=14):
    delta = series.diff().dropna()
    gain = (delta.where(delta > 0, 0)).rolling(window=period).mean()
    loss = (-delta.where(delta < 0, 0)).rolling(window=period).mean()
    rs = gain / (loss + 1e-10)
    return 100 - (100 / (1 + rs))

def add_datetime_features(df):
    df["Timestamp"] = pd.to_datetime(df["Timestamp"], unit="s")
    df["Weekday"] = df["Timestamp"].dt.weekday
    df["Day"] = df["Timestamp"].dt.day
    df["Year"] = df["Timestamp"].dt.year.astype(float)

    df["TOD"] = df["Timestamp"].dt.hour + df["Timestamp"].dt.minute/60.0

    month = df["Timestamp"].dt.month
    df["Month_Sin"] = np.sin(2 * np.pi * month / 12.0)
    df["Month_Cos"] = np.cos(2 * np.pi * month / 12.0)

    df["TOD_Sin"] = np.sin(2 * np.pi * df["TOD"] / 24.0)
    df["TOD_Cos"] = np.cos(2 * np.pi * df["TOD"] / 24.0)

    df.drop(columns=["Timestamp", "TOD"], inplace=True)
```

```

    return df

def add_features(df):
    df["SMA_10"] = df["Close"].rolling(window=10).mean()
    df["SMA_50"] = df["Close"].rolling(window=50).mean()
    df["RSI_14"] = compute_rsi(df["Close"])
    df["Return"] = df["Close"].pct_change()

    df["Middle_Band"] = df["Close"].rolling(window=20).mean()
    stddev = df["Close"].rolling(window=20).std()
    df["Upper_Band"] = df["Middle_Band"] + (2 * stddev)
    df["Lower_Band"] = df["Middle_Band"] - (2 * stddev)

    df = add_datetime_features(df)
    df["Target"] = df["Close"].shift(-1)
    return df

def create_sequences(data, seq_length):
    X, y = [], []
    for i in range(len(data) - seq_length):
        X.append(data[i : i+seq_length, :-1])
        y.append(data[i + seq_length, -1])
    return np.array(X), np.array(y)

def prepare_data(filepath, scaling_method="minmax", seq_length=30,
    val_fraction=0.15):
    df = pd.read_csv(filepath)
    df.columns = ["Timestamp", "Open", "High", "Low", "Close", "Volume",
    "Trades"]
    df = df.reset_index(drop=True)

    n = len(df)
    train_end = int(0.7 * n)
    val_size = int(val_fraction * n)
    val_end = train_end + val_size

    train_df = df.iloc[:train_end].copy()
    val_df = df.iloc[train_end:val_end].copy()
    test_df = df.iloc[val_end:].copy()

    train_df = add_features(train_df)
    val_df = add_features(val_df)
    test_df = add_features(test_df)

    train_means = train_df.mean(numeric_only=True)
    train_df.fillna(train_means, inplace=True)
    val_df.fillna(train_means, inplace=True)

```

```

test_df.fillna(train_means, inplace=True)

if scaling_method == "minmax":
    scaler = MinMaxScaler()
elif scaling_method == "standard":
    scaler = StandardScaler()
else:
    raise ValueError(f"Invalid scaling method: {scaling_method}")

exclude_cols = ["Month_Sin", "Month_Cos", "TOD_Sin", "TOD_Cos"]
features_to_scale = [c for c in train_df.columns if c not in exclude_cols]

scaler.fit(train_df[features_to_scale])
train_df[features_to_scale] = scaler.transform(train_df[features_to_scale])
val_df[features_to_scale] = scaler.transform(val_df[features_to_scale])
test_df[features_to_scale] = scaler.transform(test_df[features_to_scale])

def reorder(d):
    cols = list(d.columns)
    cols.remove("Target")
    cols.append("Target")
    return d[cols]

train_df = reorder(train_df)
val_df = reorder(val_df)
test_df = reorder(test_df)

train_data = train_df.to_numpy()
val_data = val_df.to_numpy()
test_data = test_df.to_numpy()

X_train, y_train = create_sequences(train_data, seq_length)
X_val, y_val = create_sequences(val_data, seq_length)
X_test, y_test = create_sequences(test_data, seq_length)

X_train, y_train = X_train[:-1], y_train[:-1]
X_val, y_val = X_val[:-1], y_val[:-1]
X_test, y_test = X_test[:-1], y_test[:-1]

return X_train, y_train, X_val, y_val, X_test, y_test

#####
# 2) Custom HyperModel with build(...) and fit(...)
#####
class FullSearchHyperModel(kt.HyperModel):
    def build(self, hp):

```

```

        self.scaling_method = hp.Choice("scaling_method", ["minmax",
↪"standard"])
        self.seq_length      = hp.Int("seq_length", 10, 60, step=10)
        self.val_fraction    = hp.Float("val_fraction", 0.1, 0.3, step=0.1)

        (self.X_train,
         self.y_train,
         self.X_val,
         self.y_val,
         self.X_test,
         self.y_test) = prepare_data(
            filepath="XBTUSD_720.csv",
            scaling_method=self.scaling_method,
            seq_length=self.seq_length,
            val_fraction=self.val_fraction
        )

        num_features = self.X_train.shape[2]

        model = tf.keras.Sequential()
        num_lstm_layers = hp.Int("num_lstm_layers", min_value=1, max_value=3,
↪step=1)

        for i in range(num_lstm_layers):
            units = hp.Int(f"lstm_units_{i}", 32, 128, step=32)
            return_seq = (i < num_lstm_layers - 1)
            if i == 0:
                model.add(Bidirectional(
                    LSTM(units, return_sequences=return_seq,
                        input_shape=(self.seq_length, num_features))
                ))
            else:
                model.add(LSTM(units, return_sequences=return_seq))

            dp = hp.Float(f"dropout_rate_{i}", 0.0, 0.5, step=0.1)
            model.add(Dropout(dp))

        dense_units = hp.Int("dense_units", 16, 128, step=16)
        dense_act    = hp.Choice("dense_activation", ["relu", "tanh", "sigmoid"])
        model.add(Dense(dense_units, activation=dense_act))

        model.add(Dense(1))

        optimizer_choice = hp.Choice("optimizer", ["adam", "sgd", "rmsprop"])
        learning_rate = hp.Float("learning_rate", 1e-5, 1e-2, sampling="log")

        if optimizer_choice == "adam":

```

```

        optimizer = tf.keras.optimizers.Adam(learning_rate=learning_rate)
    elif optimizer_choice == "sgd":
        optimizer = tf.keras.optimizers.SGD(learning_rate=learning_rate)
    elif optimizer_choice == "rmsprop":
        optimizer = tf.keras.optimizers.RMSprop(learning_rate=learning_rate)

    loss_choice = hp.Choice("loss", ["mse", "mae"])
    model.compile(
        optimizer=optimizer,
        loss=loss_choice,
        metrics=["mae"]
    )

    return model

def fit(self, hp, model, *args, **kwargs):
    batch_size = hp.Int("batch_size", 16, 128, step=16)

    early_stopping = tf.keras.callbacks.EarlyStopping(
        monitor='val_loss',
        patience=10,
        restore_best_weights=True
    )

    callbacks = kwargs.pop('callbacks', [])
    callbacks.append(early_stopping)

    return model.fit(
        self.X_train, self.y_train,
        validation_data=(self.X_val, self.y_val),
        epochs=100,
        batch_size=batch_size,
        callbacks=callbacks,
        verbose=0,
        **kwargs
    )

#####
# 3) Use Bayesian Optimization Tuner
#####
from keras_tuner.tuners import BayesianOptimization

tuner = BayesianOptimization(
    hypermodel=FullSearchHyperModel(),
    objective="val_loss",
    max_trials=100,
    seed=42,

```

```

    project_name="btc_lstm_full_tuning_allinone",
    overwrite=True
)

tuner.search()

```

Trial 100 Complete [00h 00m 48s]
val_loss: 0.004845061339437962

Best val_loss So Far: 0.0009701328235678375
Total elapsed time: 01h 28m 53s

```

[ ]: #####
# 4) Results & Summaries
#####
tuner.results_summary()

# Gather all trials
all_trials = tuner.oracle.get_best_trials(num_trials=20)
records = []
for t in all_trials:
    hps = t.hyperparameters.values
    loss = t.metrics.get_last_value("val_loss")
    row = {**hps, "val_loss": loss}
    records.append(row)

df = pd.DataFrame(records).sort_values("val_loss")
print("\nTop Trials (Sorted by val_loss):")
print(df.head(5))

#####
# 5) Evaluate on the Test Set With the Best Model
#####
best_hp = tuner.get_best_hyperparameters(num_trials=1)[0]
print("\nBEST HYPERPARAM COMBINATION:")
print(best_hp.values)

best_model = tuner.get_best_models(num_models=1)[0]
best_model.summary()

# Re-run data prep with best hyperparams
best_scaling = best_hp.get("scaling_method")
best_seq_len = best_hp.get("seq_length")
best_val_frac = best_hp.get("val_fraction")

X_train, y_train, X_val, y_val, X_test, y_test = prepare_data(
    "XBTUSD_720.csv",

```

```

        scaling_method=best_scaling,
        seq_length=best_seq_len,
        val_fraction=best_val_frac
    )

    # Add EarlyStopping for final training
    early_stopping = tf.keras.callbacks.EarlyStopping(
        monitor='val_loss',
        patience=10,
        restore_best_weights=True
    )

    # Re-train with early stopping (epochs no longer in hyperparams)
    history = best_model.fit(
        X_train, y_train,
        validation_data=(X_val, y_val),
        epochs=100, # Fixed high number with early stopping
        batch_size=best_hp.get("batch_size"),
        callbacks=[early_stopping],
        verbose=1
    )

    # Evaluate on test set
    test_loss, test_mae = best_model.evaluate(X_test, y_test, verbose=1)
    print("\nFinal Test Loss:", test_loss)
    print("Final Test MAE:", test_mae)

    #####
    # 6) Visualization & Performance Analysis
    #####

    # 1. Training vs Validation Loss
    plt.figure(figsize=(10, 5))
    plt.plot(history.history["loss"], label="Train Loss")
    plt.plot(history.history["val_loss"], label="Validation Loss")
    plt.xlabel("Epoch")
    plt.ylabel("Loss (MSE)")
    plt.title("Training vs Validation Loss Curve")
    plt.legend()
    plt.grid(True)
    plt.tight_layout()
    plt.show()

    # 2. Predictions vs True Values (Time Series)
    preds = best_model.predict(X_test).flatten()

    plt.figure(figsize=(12, 5))

```

```

plt.plot(y_test, label="True", alpha=0.6)
plt.plot(preds, label="Predicted", alpha=0.6)
plt.title("Predicted vs True Close Price (1 Step Ahead)")
plt.xlabel("Time Step")
plt.ylabel("Normalized Close Price")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

# 3. Scatter Plot with Correlation
plt.figure(figsize=(6, 6))
plt.scatter(y_test, preds, alpha=0.5)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--')
plt.xlabel("True Values")
plt.ylabel("Predicted Values")
plt.title("True vs Predicted Values (Scatter)")
plt.grid(True)
plt.tight_layout()
plt.show()

correlation = np.corrcoef(y_test, preds)[0, 1]
print("Prediction-Target Correlation: {:.4f}".format(correlation))

#####
# 7) Baseline Comparison, Skill Score, and Dollar/Percentage Errors
#####

import numpy as np
from sklearn.metrics import mean_squared_error, mean_absolute_error

# -----
# A. NAÏVE FORECAST IN THE SCALED DOMAIN
# -----

# We'll shift y_test by 1 step. That means naive_preds[t] = y_test[t-1].
naive_preds = np.zeros_like(y_test)
naive_preds[1:] = y_test[:-1] # shift by 1
naive_preds[0] = y_test[0]    # or could be set to 0 or np.nan

# MSE & MAE of naive forecast (scaled domain)
naive_mse = mean_squared_error(y_test, naive_preds)
naive_mae = mean_absolute_error(y_test, naive_preds)

# MSE & MAE of model forecast (scaled domain) - we already have `test_loss`
↳ (usually MSE)

```

```

# and `test_mae`. But let's re-confirm with scikit-learn so we can do a skill_
↪score:

model_mse_scaled = mean_squared_error(y_test, preds)
model_mae_scaled = mean_absolute_error(y_test, preds)

print("\n--- Naïve Forecast (Scaled) ---")
print("Naive MSE:", naive_mse)
print("Naive MAE:", naive_mae)

print("\n--- Model (Scaled) ---")
print("Model MSE:", model_mse_scaled)
print("Model MAE:", model_mae_scaled)

# -----
# B. Skill Score (for MSE)
# -----
# skill_score = 1 - (model_error / naive_error)
if naive_mse > 0:
    skill_score = 1 - (model_mse_scaled / naive_mse)
else:
    skill_score = None # in case naive_mse = 0
print("\nSkill Score (1 - MSE_model/MSE_naive):", skill_score)

# -----
# C. UNDO SCALING: GET DOLLAR ERRORS
# -----
# We need to invert the scaling. Let's assume we want to invert only the_
↪'Close'/'Target' column.
# 1) Reload your CSV so we can get an actual scaler for the 'Close' only.
# Or if you still have the 'scaler' from prepare_data, use that.
# But we have to be careful because your code scales multiple columns.
#
# Easiest approach: We'll create a separate scaler JUST for the 'Close' column_
↪for demonstration.

df_original = pd.read_csv("XBTUSD_720.csv")
df_original.columns = ["Timestamp", "Open", "High", "Low", "Close", "Volume",_
↪"Trades"]

# Let's focus on the test subset for that 'Close' column only:
n = len(df_original)
train_end = int(0.7 * n)
val_size = int(0.15 * n)
val_end = train_end + val_size

test_df = df_original.iloc[val_end:].copy()

```

```

# Remove the last row if we had done so in prepare_data:
if len(test_df) > len(y_test):
    test_df = test_df.iloc[:len(y_test)]

# Now, fit a MinMax scaler or Standard scaler to ONLY the train portion of
    ↪ 'Close',
# or if you want to replicate exactly, do the same approach as your code's
    ↪ "prepare_data".
# For a quick demo, let's do a naive MinMax fit on the entire 'Close' just to
    ↪ show the method.

close_scaler = MinMaxScaler()
# Fit on entire 'Close' just for demonstration (not strictly correct vs. your
    ↪ code).
close_scaler.fit(df_original[["Close"]])

# We can invert model predictions:
preds_unscaled = close_scaler.inverse_transform(preds.reshape(-1,1)).flatten()
naive_unscaled = close_scaler.inverse_transform(naive_preds.reshape(-1,1)).
    ↪ flatten()
y_test_unscaled = close_scaler.inverse_transform(y_test.reshape(-1,1)).flatten()

# Now compute errors in DOLLAR terms
model_mse_dollars = mean_squared_error(y_test_unscaled, preds_unscaled)
model_mae_dollars = mean_absolute_error(y_test_unscaled, preds_unscaled)
naive_mse_dollars = mean_squared_error(y_test_unscaled, naive_unscaled)
naive_mae_dollars = mean_absolute_error(y_test_unscaled, naive_unscaled)

print("\n--- Unscaled (Dollars) ---")
print("Model MSE (dollars):", model_mse_dollars)
print("Model MAE (dollars):", model_mae_dollars)
print("Naive MSE (dollars):", naive_mse_dollars)
print("Naive MAE (dollars):", naive_mae_dollars)

# -----
# D. MAE as Percentage
# -----
# We'll do: MAE% = mean(|pred - true| / true) * 100
# In practice, watch out for any zero or near-zero target values.

pct_errors = np.abs(y_test_unscaled - preds_unscaled) / np.abs(y_test_unscaled)
pct_errors = pct_errors[~np.isnan(pct_errors)] # drop any NaNs if 0 in
    ↪ denominator
mae_pct = np.mean(pct_errors) * 100

```

```

print(f"\nMean Absolute Percentage Error (Model): {mae_pct:.2f}%")

#####
# 8) Up/Down Classification Task - Metrics & Visualization
#####

from sklearn.metrics import (
    accuracy_score, precision_score, recall_score, f1_score,
    confusion_matrix, classification_report, roc_auc_score
)
import seaborn as sns

# -- Step 1: Direction Labels (1 = up, 0 = down) --
def compute_direction(arr):
    return np.where(np.diff(arr, prepend=arr[0]) > 0, 1, 0)

true_dir = compute_direction(y_test_unscaled)
pred_dir = compute_direction(preds_unscaled)

# -- Step 2: Compute Metrics --
acc = accuracy_score(true_dir, pred_dir)
prec = precision_score(true_dir, pred_dir)
rec = recall_score(true_dir, pred_dir)
f1 = f1_score(true_dir, pred_dir)
try:
    auc = roc_auc_score(true_dir, pred_dir)
except ValueError:
    auc = None

print("\n--- Up/Down Classification Metrics ---")
print(f"Accuracy : {acc:.4f}")
print(f"Precision: {prec:.4f}")
print(f"Recall : {rec:.4f}")
print(f"F1 Score : {f1:.4f}")
print(f"ROC AUC : {auc if auc is not None else 'N/A'}")

# -- Step 3: Confusion Matrix --
cm = confusion_matrix(true_dir, pred_dir)
labels = ["Down", "Up"]

plt.figure(figsize=(5, 4))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues", xticklabels=labels,
            yticklabels=labels)
plt.title("Confusion Matrix: Up vs Down")
plt.xlabel("Predicted Direction")
plt.ylabel("True Direction")

```

```

plt.tight_layout()
plt.show()

# -- Step 4: Classification Report --
print("\nClassification Report:")
print(classification_report(true_dir, pred_dir, target_names=labels))

# -- Step 5: Direction Over Time Plot --
plt.figure(figsize=(12, 4))
plt.plot(true_dir, label="True Direction", linestyle="--", alpha=0.6)
plt.plot(pred_dir, label="Predicted Direction", alpha=0.6)
plt.title("Direction Prediction Over Time")
plt.xlabel("Time Step")
plt.ylabel("Direction (0=Down, 1=Up)")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()

# -- Step 6: Error Heatmap (Optional) --
error_mask = true_dir != pred_dir
plt.figure(figsize=(12, 2))
plt.imshow(error_mask.reshape(1, -1), cmap="coolwarm", aspect="auto")
plt.title("Misclassification Heatmap (Red = Error)")
plt.yticks([])
plt.xlabel("Time Step")
plt.tight_layout()
plt.show()

```

Results summary
Results in ./btc_lstm_full_tuning_allinone
Showing 10 best trials
Objective(name="val_loss", direction="min")

Trial 066 summary
Hyperparameters:
scaling_method: minmax
seq_length: 10
val_fraction: 0.2
num_lstm_layers: 1
lstm_units_0: 64
dropout_rate_0: 0.2
dense_units: 80
dense_activation: tanh
optimizer: rmsprop
learning_rate: 0.0003065440577279104
loss: mse
batch_size: 16

lstm_units_1: 64
dropout_rate_1: 0.4
lstm_units_2: 96
dropout_rate_2: 0.4
Score: 0.0009701328235678375

Trial 074 summary

Hyperparameters:

scaling_method: minmax
seq_length: 30
val_fraction: 0.2
num_lstm_layers: 1
lstm_units_0: 32
dropout_rate_0: 0.2
dense_units: 80
dense_activation: sigmoid
optimizer: adam
learning_rate: 0.006246948989104125
loss: mse
batch_size: 16
lstm_units_1: 32
dropout_rate_1: 0.4
lstm_units_2: 32
dropout_rate_2: 0.4
Score: 0.0011496879160404205

Trial 077 summary

Hyperparameters:

scaling_method: minmax
seq_length: 40
val_fraction: 0.2
num_lstm_layers: 1
lstm_units_0: 32
dropout_rate_0: 0.0
dense_units: 64
dense_activation: tanh
optimizer: rmsprop
learning_rate: 0.01
loss: mse
batch_size: 80
lstm_units_1: 128
dropout_rate_1: 0.4
lstm_units_2: 128
dropout_rate_2: 0.4
Score: 0.0013798980507999659

Trial 050 summary

Hyperparameters:

scaling_method: minmax
seq_length: 30
val_fraction: 0.2
num_lstm_layers: 2
lstm_units_0: 64
dropout_rate_0: 0.30000000000000004
dense_units: 64
dense_activation: tanh
optimizer: adam
learning_rate: 0.001928338104221399
loss: mse
batch_size: 16
lstm_units_1: 96
dropout_rate_1: 0.2
lstm_units_2: 96
dropout_rate_2: 0.1
Score: 0.0014566172612830997

Trial 081 summary

Hyperparameters:

scaling_method: minmax
seq_length: 60
val_fraction: 0.2
num_lstm_layers: 1
lstm_units_0: 96
dropout_rate_0: 0.0
dense_units: 64
dense_activation: relu
optimizer: adam
learning_rate: 0.01
loss: mse
batch_size: 80
lstm_units_1: 64
dropout_rate_1: 0.0
lstm_units_2: 128
dropout_rate_2: 0.1
Score: 0.001674074330367148

Trial 056 summary

Hyperparameters:

scaling_method: minmax
seq_length: 50
val_fraction: 0.1
num_lstm_layers: 1
lstm_units_0: 64
dropout_rate_0: 0.1
dense_units: 32
dense_activation: relu

optimizer: rmsprop
learning_rate: 0.01
loss: mse
batch_size: 64
lstm_units_1: 96
dropout_rate_1: 0.0
lstm_units_2: 128
dropout_rate_2: 0.0
Score: 0.0022213265765458345

Trial 019 summary
Hyperparameters:
scaling_method: minmax
seq_length: 10
val_fraction: 0.1
num_lstm_layers: 2
lstm_units_0: 96
dropout_rate_0: 0.0
dense_units: 32
dense_activation: tanh
optimizer: rmsprop
learning_rate: 0.00047865000925308724
loss: mse
batch_size: 112
lstm_units_1: 64
dropout_rate_1: 0.4
lstm_units_2: 128
dropout_rate_2: 0.4
Score: 0.0023822637740522623

Trial 004 summary
Hyperparameters:
scaling_method: minmax
seq_length: 30
val_fraction: 0.1
num_lstm_layers: 1
lstm_units_0: 96
dropout_rate_0: 0.0
dense_units: 128
dense_activation: sigmoid
optimizer: rmsprop
learning_rate: 0.0013553491857069859
loss: mse
batch_size: 32
lstm_units_1: 64
dropout_rate_1: 0.4
lstm_units_2: 128
dropout_rate_2: 0.2

Score: 0.0024807711597532034

Trial 057 summary

Hyperparameters:

scaling_method: minmax

seq_length: 30

val_fraction: 0.1

num_lstm_layers: 1

lstm_units_0: 96

dropout_rate_0: 0.1

dense_units: 48

dense_activation: relu

optimizer: adam

learning_rate: 0.003840651434282233

loss: mse

batch_size: 48

lstm_units_1: 96

dropout_rate_1: 0.30000000000000004

lstm_units_2: 128

dropout_rate_2: 0.2

Score: 0.002551008015871048

Trial 014 summary

Hyperparameters:

scaling_method: minmax

seq_length: 40

val_fraction: 0.1

num_lstm_layers: 1

lstm_units_0: 64

dropout_rate_0: 0.1

dense_units: 96

dense_activation: tanh

optimizer: rmsprop

learning_rate: 0.007910060285471268

loss: mse

batch_size: 112

lstm_units_1: 32

dropout_rate_1: 0.4

lstm_units_2: 64

dropout_rate_2: 0.4

Score: 0.0026517719961702824

Top Trials (Sorted by val_loss):

	scaling_method	seq_length	val_fraction	num_lstm_layers	lstm_units_0	\
0	minmax	10	0.2	1	64	
1	minmax	30	0.2	1	32	
2	minmax	40	0.2	1	32	
3	minmax	30	0.2	2	64	

	minmax	60	0.2	1	96	
	dropout_rate_0	dense_units	dense_activation	optimizer	learning_rate	loss \
0	0.2	80	tanh	rmsprop	0.000307	mse
1	0.2	80	sigmoid	adam	0.006247	mse
2	0.0	64	tanh	rmsprop	0.010000	mse
3	0.3	64	tanh	adam	0.001928	mse
4	0.0	64	relu	adam	0.010000	mse

	batch_size	lstm_units_1	dropout_rate_1	lstm_units_2	dropout_rate_2 \
0	16	64	0.4	96	0.4
1	16	32	0.4	32	0.4
2	80	128	0.4	128	0.4
3	16	96	0.2	96	0.1
4	80	64	0.0	128	0.1

	val_loss
0	0.000970
1	0.001150
2	0.001380
3	0.001457
4	0.001674

BEST HYPERPARAM COMBINATION:

```
{'scaling_method': 'minmax', 'seq_length': 10, 'val_fraction': 0.2,
'num_lstm_layers': 1, 'lstm_units_0': 64, 'dropout_rate_0': 0.2, 'dense_units':
80, 'dense_activation': 'tanh', 'optimizer': 'rmsprop', 'learning_rate':
0.0003065440577279104, 'loss': 'mse', 'batch_size': 16, 'lstm_units_1': 64,
'dropout_rate_1': 0.4, 'lstm_units_2': 96, 'dropout_rate_2': 0.4}
```

/usr/local/lib/python3.11/dist-packages/keras/src/layers/rnn/rnn.py:200:

UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

```
super().__init__(**kwargs)
```

/usr/local/lib/python3.11/dist-packages/keras/src/saving/saving_lib.py:757:

UserWarning: Skipping variable loading for optimizer 'rmsprop', because it has 2 variables whereas the saved optimizer has 12 variables.

```
saveable.load_own_variables(weights_store.get(inner_path))
```

Model: "sequential"

Layer (type)	Output Shape
↳ Param #	
bidirectional (Bidirectional)	(None, 128)
↳ 43,520	

```

dropout (Dropout)                                (None, 128)
↳ 0

dense (Dense)                                      (None, 80)
↳ 10,320

dense_1 (Dense)                                    (None, 1)
↳ 81

```

Total params: 53,921 (210.63 KB)

Trainable params: 53,921 (210.63 KB)

Non-trainable params: 0 (0.00 B)

```

Epoch 1/100
347/347          4s 7ms/step -
loss: 4.6562e-04 - mae: 0.0135 - val_loss: 0.0012 - val_mae: 0.0232
Epoch 2/100
347/347          2s 6ms/step -
loss: 4.5167e-04 - mae: 0.0131 - val_loss: 0.0012 - val_mae: 0.0259
Epoch 3/100
347/347          2s 6ms/step -
loss: 4.0778e-04 - mae: 0.0123 - val_loss: 0.0010 - val_mae: 0.0219
Epoch 4/100
347/347          2s 6ms/step -
loss: 3.9662e-04 - mae: 0.0119 - val_loss: 0.0019 - val_mae: 0.0362
Epoch 5/100
347/347          2s 6ms/step -
loss: 4.4859e-04 - mae: 0.0124 - val_loss: 0.0012 - val_mae: 0.0251
Epoch 6/100
347/347          2s 6ms/step -
loss: 3.8142e-04 - mae: 0.0121 - val_loss: 0.0012 - val_mae: 0.0247
Epoch 7/100
347/347          2s 6ms/step -
loss: 3.8258e-04 - mae: 0.0119 - val_loss: 0.0039 - val_mae: 0.0490
Epoch 8/100
347/347          2s 6ms/step -
loss: 3.9158e-04 - mae: 0.0120 - val_loss: 0.0011 - val_mae: 0.0245
Epoch 9/100
347/347          2s 6ms/step -
loss: 4.1453e-04 - mae: 0.0122 - val_loss: 0.0015 - val_mae: 0.0289
Epoch 10/100

```

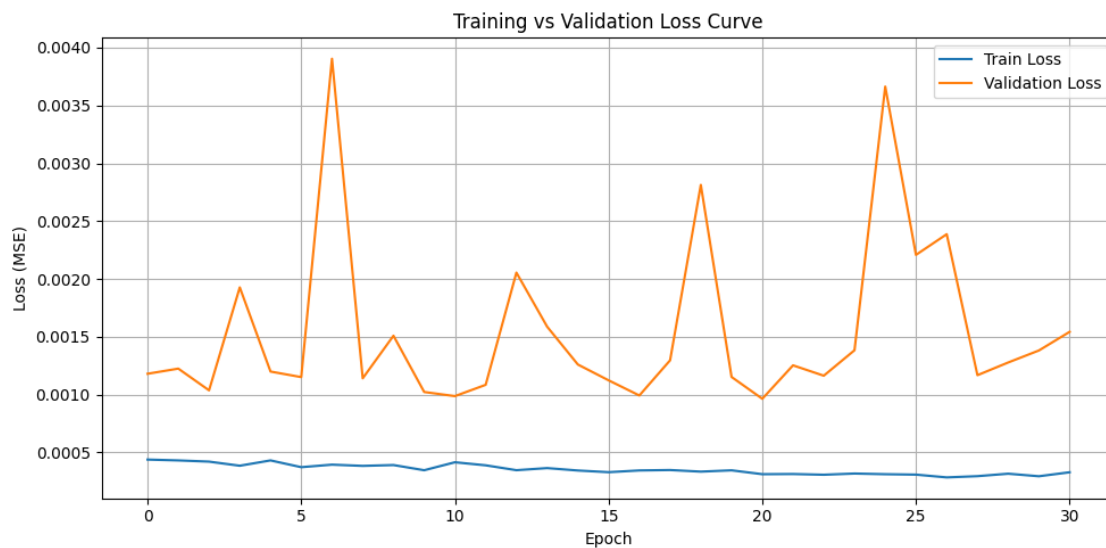
347/347 2s 6ms/step -
 loss: 3.8347e-04 - mae: 0.0124 - val_loss: 0.0010 - val_mae: 0.0241
 Epoch 11/100
 347/347 2s 6ms/step -
 loss: 4.4404e-04 - mae: 0.0129 - val_loss: 9.8714e-04 - val_mae: 0.0230
 Epoch 12/100
 347/347 2s 6ms/step -
 loss: 4.5031e-04 - mae: 0.0127 - val_loss: 0.0011 - val_mae: 0.0234
 Epoch 13/100
 347/347 2s 6ms/step -
 loss: 3.4522e-04 - mae: 0.0113 - val_loss: 0.0021 - val_mae: 0.0307
 Epoch 14/100
 347/347 2s 6ms/step -
 loss: 3.7981e-04 - mae: 0.0111 - val_loss: 0.0016 - val_mae: 0.0306
 Epoch 15/100
 347/347 2s 6ms/step -
 loss: 3.0668e-04 - mae: 0.0107 - val_loss: 0.0013 - val_mae: 0.0270
 Epoch 16/100
 347/347 2s 6ms/step -
 loss: 3.7833e-04 - mae: 0.0114 - val_loss: 0.0011 - val_mae: 0.0229
 Epoch 17/100
 347/347 2s 6ms/step -
 loss: 3.3022e-04 - mae: 0.0111 - val_loss: 9.9260e-04 - val_mae: 0.0221
 Epoch 18/100
 347/347 2s 6ms/step -
 loss: 3.8610e-04 - mae: 0.0118 - val_loss: 0.0013 - val_mae: 0.0289
 Epoch 19/100
 347/347 2s 6ms/step -
 loss: 3.4126e-04 - mae: 0.0112 - val_loss: 0.0028 - val_mae: 0.0410
 Epoch 20/100
 347/347 2s 6ms/step -
 loss: 3.8125e-04 - mae: 0.0113 - val_loss: 0.0012 - val_mae: 0.0240
 Epoch 21/100
 347/347 2s 6ms/step -
 loss: 3.0731e-04 - mae: 0.0110 - val_loss: 9.6416e-04 - val_mae: 0.0220
 Epoch 22/100
 347/347 2s 6ms/step -
 loss: 3.1617e-04 - mae: 0.0108 - val_loss: 0.0013 - val_mae: 0.0260
 Epoch 23/100
 347/347 2s 6ms/step -
 loss: 2.7218e-04 - mae: 0.0103 - val_loss: 0.0012 - val_mae: 0.0248
 Epoch 24/100
 347/347 2s 6ms/step -
 loss: 3.5089e-04 - mae: 0.0110 - val_loss: 0.0014 - val_mae: 0.0263
 Epoch 25/100
 347/347 2s 6ms/step -
 loss: 2.8842e-04 - mae: 0.0102 - val_loss: 0.0037 - val_mae: 0.0501
 Epoch 26/100

```

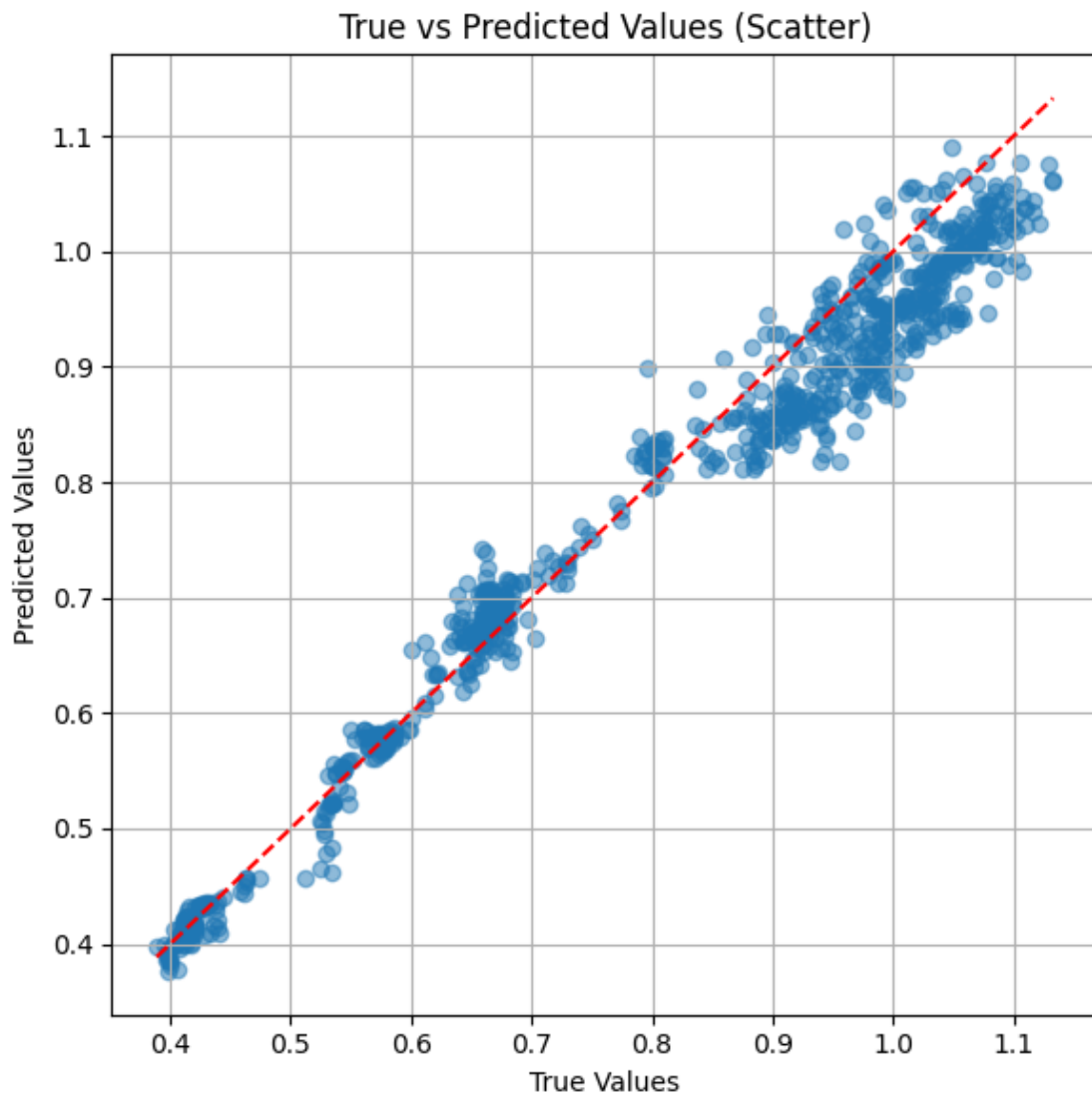
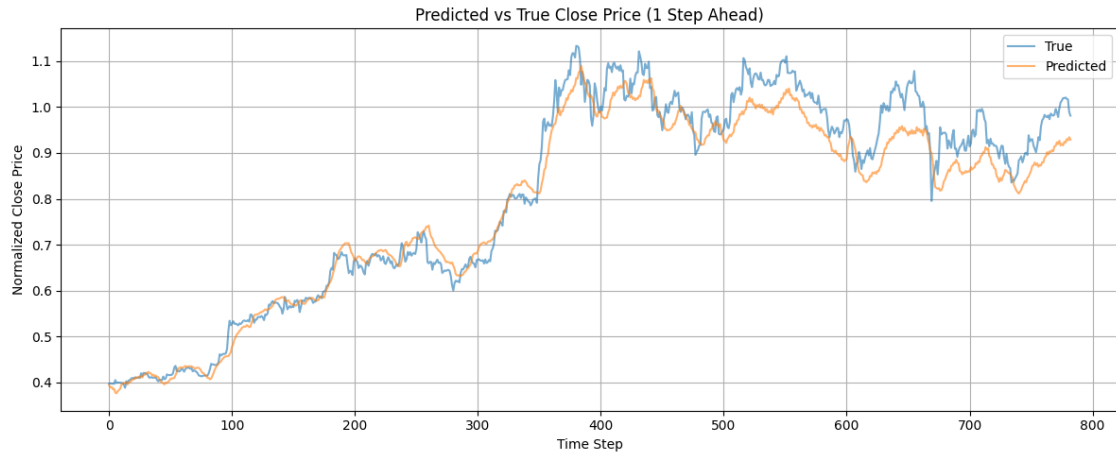
347/347          2s 6ms/step -
loss: 3.4424e-04 - mae: 0.0107 - val_loss: 0.0022 - val_mae: 0.0393
Epoch 27/100
347/347          2s 6ms/step -
loss: 2.8202e-04 - mae: 0.0103 - val_loss: 0.0024 - val_mae: 0.0349
Epoch 28/100
347/347          2s 6ms/step -
loss: 2.8796e-04 - mae: 0.0103 - val_loss: 0.0012 - val_mae: 0.0255
Epoch 29/100
347/347          2s 6ms/step -
loss: 3.0109e-04 - mae: 0.0106 - val_loss: 0.0013 - val_mae: 0.0273
Epoch 30/100
347/347          2s 6ms/step -
loss: 2.6784e-04 - mae: 0.0101 - val_loss: 0.0014 - val_mae: 0.0247
Epoch 31/100
347/347          2s 7ms/step -
loss: 3.3851e-04 - mae: 0.0105 - val_loss: 0.0015 - val_mae: 0.0305
25/25            0s 4ms/step - loss:
0.0010 - mae: 0.0217

```

Final Test Loss: 0.0022574583999812603
Final Test MAE: 0.036836281418800354



25/25 0s 9ms/step



Prediction-Target Correlation: 0.9866

--- Naïve Forecast (Scaled) ---

Naive MSE: 0.0002588658692728566

Naive MAE: 0.010454150906689027

--- Model (Scaled) ---

Model MSE: 0.002257458503706119

Model MAE: 0.036836280966843646

Skill Score (1 - MSE_model/MSE_naive): -7.72057220230394

--- Unscaled (Dollars) ---

Model MSE (dollars): 12027191.350985877

Model MAE (dollars): 2688.7350038221025

Naive MSE (dollars): 1379174.605614208

Naive MAE (dollars): 763.0640513640828

Mean Absolute Percentage Error (Model): 4.17%

--- Up/Down Classification Metrics ---

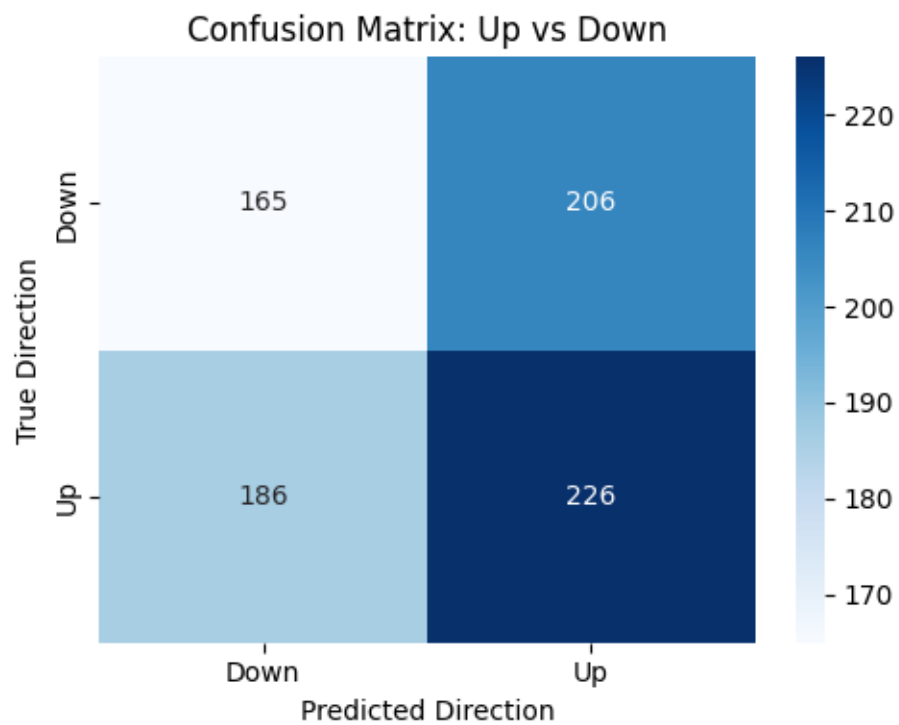
Accuracy : 0.4994

Precision: 0.5231

Recall : 0.5485

F1 Score : 0.5355

ROC AUC : 0.4966438123151807



Classification Report:

	precision	recall	f1-score	support
Down	0.47	0.44	0.46	371
Up	0.52	0.55	0.54	412
accuracy			0.50	783
macro avg	0.50	0.50	0.50	783
weighted avg	0.50	0.50	0.50	783

